

PRACTICAL – 3

Q) Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }

    else if (pid == 0) {
        // Child Process
        printf("\n==== Child Process =====\n");
        printf("Child PID : %d\n", getpid());
```

```
printf("Parent PID : %d\n", getppid());

printf("Child is running...\n");
sleep(10);  // Keeps child alive for observation

printf("Child process completed.\n");
}

else {
    // Parent Process
    printf("\n===== Parent Process =====\n");
    printf("Parent PID : %d\n", getpid());
    printf("Child PID : %d\n", pid);

    printf("Parent waiting for child...\n");

    wait(NULL);

    printf("Child terminated.\n");
    printf("Parent process completed.\n");
}

return 0;
}
```

Output:

```
ritwik@DESKTOP-PK3LAMN:~/OSLab/Practicactical1$ nano program3.c
ritwik@DESKTOP-PK3LAMN:~/OSLab/Practicactical1$ gcc program3.c -o program3
ritwik@DESKTOP-PK3LAMN:~/OSLab/Practicactical1$ ./program3

===== Parent Process =====
Parent PID : 859
Child PID  : 860
Parent waiting for child...

===== Child Process =====
Child PID  : 860
Parent PID : 859
Child is running...
Child process completed.
Child terminated.
Parent process completed.
```